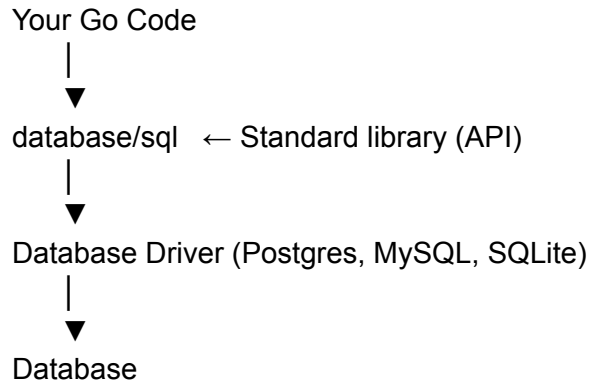


No. **database/sql** is *not* an ORM.

It is Go's **standard database package**, which provides a common interface for working with SQL databases.

Think of it like this:



For example:

```
import (  
    "database/sql"  
    _ "github.com/lib/pq" // PostgreSQL driver  
)
```

Here:

- **database/sql** provides the API (**sql.DB**, **Query**, **Exec**, **Begin**, etc.).
  - **github.com/lib/pq** is the PostgreSQL driver that actually communicates with PostgreSQL.
- 

## What is an ORM?

An **ORM (Object-Relational Mapper)** maps Go structs to database tables so you don't write much SQL yourself.

Example with an ORM:

```
type User struct {  
    ID int
```

```
    Name string
}
```

```
db.Create(&User{Name: "Suyash"})
```

The ORM generates SQL like:

```
INSERT INTO users (name) VALUES ('Suyash');
```

You don't write the SQL.

---

## What do you do with **database/sql**?

You write the SQL yourself.

```
db.Exec(
    "INSERT INTO users(name) VALUES(?)",
    "Suyash",
)
```

or

```
rows, err := db.Query(
    "SELECT id, name FROM users",
)
```

You are responsible for:

- Writing SQL
  - Scanning rows into structs
  - Handling transactions
  - Managing prepared statements
- 

## Comparison

| Feature                    | database/sql | ORM (like GORM)           |
|----------------------------|--------------|---------------------------|
| Write SQL yourself         | ✓ Yes        | ✗ Usually no              |
| Maps structs automatically | ✗ No         | ✓ Yes                     |
| More control               | ✓ Yes        | Less                      |
| Less abstraction           | ✓ Yes        | ✗ No                      |
| Performance                | Excellent    | Good (with some overhead) |

---

## Why do many Go developers prefer database/sql?

Go's philosophy is simplicity and explicitness. Many backend teams prefer writing SQL directly because:

- SQL is powerful and expressive.
- Queries are easier to optimize.
- There is less "magic" than with an ORM.
- Performance is predictable.

This is why you'll often see production Go projects use:

- database/sql
- SQL drivers (PostgreSQL, MySQL, SQLite)
- Query builders or code generators like sqlc or Squirrel instead of a full ORM.

So, **database/sql** is the standard database API in Go—not an ORM.

Absolutely. This is one of the most important concepts in backend development. Once you understand this, **database/sql** will make much more sense.

---

## Imagine this real-life example

Suppose you're in Japan.

You only speak **English**.

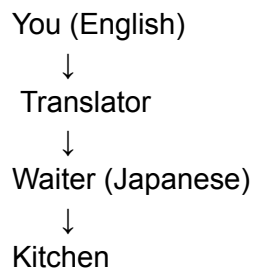
The waiter only speaks **Japanese**.

You want to order pizza.

Can you directly talk to the waiter?

✗ No.

You need a **translator**.

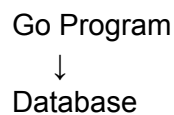


The translator understands both languages.

---

## Now replace this with a database

Suppose your Go program wants to save a student.



Can Go directly talk to PostgreSQL?

✗ No.

Why?

Because PostgreSQL speaks its own database protocol.

Go doesn't know that protocol.

So something has to translate.

---

## This is where the driver comes in

Go Program



Driver



PostgreSQL

The driver is the translator.

For PostgreSQL:

[github.com/lib/pq](https://github.com/lib/pq)

or

[github.com/jackc/pgx](https://github.com/jackc/pgx)

For MySQL:

[github.com/go-sql-driver/mysql](https://github.com/go-sql-driver/mysql)

For SQLite:

[github.com/mattn/go-sqlite3](https://github.com/mattn/go-sqlite3)

Each driver knows how to communicate with its specific database.

---

## But then what is **database/sql**?

Imagine every restaurant has a different translator.

Japanese Translator

French Translator

German Translator

Each translator speaks differently.

If you had to learn every translator's style, your code would change every time you switched databases.

That would be terrible.

So Go provides a common API.

Your Code



database/sql



Driver



Database

---

## Think of **database/sql** as an interface

Instead of talking to the driver directly,

you always talk to

database/sql

It provides functions like

`db.Query()`

`db.Exec()`

`db.Begin()`

`db.Ping()`

```
db.Close()
```

Every driver implements these operations.

---

## Complete flow

Let's say you write

```
db.Exec(  
    "INSERT INTO students(name) VALUES($1)",  
    "Suyash",  
)
```

What actually happens?

Step 1

Your code calls

```
db.Exec(...)
```

↓

Step 2

`database/sql` receives it

↓

Step 3

It forwards the request to the PostgreSQL driver

↓

Step 4

The driver converts it into PostgreSQL's wire protocol (the format PostgreSQL expects over the network).

↓

Step 5

PostgreSQL receives it.

↓

Step 6

PostgreSQL executes

```
INSERT INTO students(name)
VALUES('Suyash');
```

↓

Step 7

Database says

Success!

↓

Step 8

Driver receives the response.

↓

Step 9

Driver converts the response into Go values.

↓

Step 10

`database/sql` returns

`sql.Result`

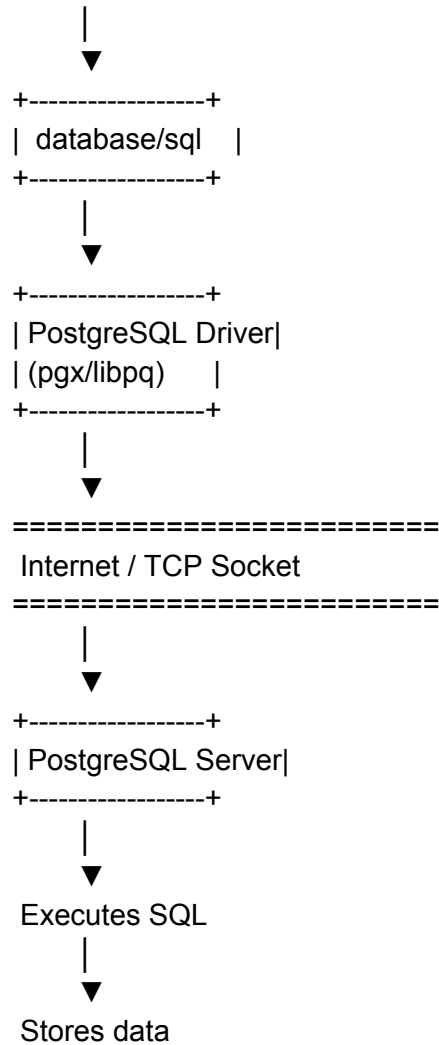
to your Go program.

---

# Picture the entire journey

You write

```
db.Exec("INSERT ...")
```



---

## Why can't `database/sql` talk directly to PostgreSQL?

Because every database speaks differently.

PostgreSQL protocol:

Message Type A

Message Type B

Message Type C

MySQL protocol:

Packet X

Packet Y

Packet Z

SQLite doesn't even use a network—it reads and writes a local file.

So Go would have to implement every database protocol itself.

Instead, Go says:

"We'll define a standard API (`database/sql`). Each database driver will implement the details."

---

## What happens when you import a driver?

You often see this:

```
import (  
    "database/sql"  
  
    _ "github.com/lib/pq"  
)
```

Notice the underscore (`_`).

That means:

"Run the package's initialization code, but I won't call it directly."

During initialization, the driver registers itself with `database/sql`.

You can think of it like this:

Driver starts

↓

database/sql says

"Who are you?"

↓

Driver says

"I'm PostgreSQL."

↓

database/sql stores it

map["postgres"] = postgresDriver

Later, when you write:

```
db, err := sql.Open("postgres", connString)
```

`database/sql` looks up the registered `"postgres"` driver and asks it to create a connection.

---

## Analogy with HTTP

You've learned:

`net/http`

It gives you

`http.ListenAndServe()`

`http.NewServeMux()`

`http.Request`

`http.ResponseWriter`

Different web servers can work with this API.

Similarly,

`database/sql`

gives you

`sql.DB`

`sql.Tx`

`sql.Rows`

`sql.Stmt`

Different database drivers work with this API.

---

## One important clarification

Many beginners think:

"`database/sql` connects to PostgreSQL."

That's **not quite true**.

A more accurate picture is:

- **`database/sql` defines the common database API and manages things like connection pooling.**
- **The driver performs the actual communication with the database.**
- **The database executes the SQL and stores or retrieves the data.**

If you keep that separation in mind, the architecture becomes much easier to understand.

😓 Don't worry. This is one of those Go concepts that almost everyone finds confusing at first. Let's go **very slowly**.

---

## First, look at this import

```
import (  
    "database/sql"  
    _ "github.com/mattn/go-sqlite3"  
)
```

The question is:

**Why are we importing `github.com/mattn/go-sqlite3` if we never use it?**

That's exactly the right question.

---

## Normally, imports work like this

Suppose you write

```
import "fmt"
```

Then you use it.

```
fmt.Println("Hello")
```

Easy.

---

## But here...

```
import _ "github.com/mattn/go-sqlite3"
```

You never do

```
sqlite3.Connect()
```

or

```
sqlite3.Open()
```

Nothing.

So why import it?

---

## The answer is: because we only want its `init()` function to run.

Every Go package **can** have an `init()` function.

Example:

```
package animal
```

```
import "fmt"
```

```
func init() {  
    fmt.Println("Animal package initialized")  
}
```

---

Now imagine

```
package main
```

```
import _ "animal"
```

```
func main() {  
    fmt.Println("Main")  
}
```

Output:

Animal package initialized  
Main

Notice something?

We never called

`animal.init()`

Go called it automatically.

---

## Go automatically runs `init()`

The order is:

Program starts

↓

Imports package

↓

Runs `init()`

↓

Runs `main()`

You never call `init()` yourself.

---

## Now let's come back to SQLite

Inside the SQLite driver (simplified), there is something like this:

```
package sqlite3

func init() {
    sql.Register("sqlite3", &SQLiteDriver{})
}
```

You never see this code because it's inside the driver package.

---

## What is `sql.Register()`?

Think of `database/sql` as maintaining a directory of available drivers.

Initially:

`database/sql`

Registered Drivers

(empty)

---

When the SQLite driver's `init()` runs:

```
sql.Register("sqlite3", &SQLiteDriver{})
```

it tells `database/sql`:

"Hi, I'm the SQLite driver. If anyone asks for `"sqlite3"`, use me."

Now the directory looks like:

`database/sql`

Registered Drivers

`sqlite3`

↓

SQLiteDriver

---

## Later you write

```
db, err := sql.Open("sqlite3", "students.db")
```

What happens?

`database/sql` thinks:

User wants `sqlite3`.

↓

Do I know `sqlite3`?

↓

Yes!

↓

I have a registered driver.

↓

I'll use that driver.

Without registration, it has no idea what "`sqlite3`" means.

---

## What if you DON'T import the driver?

Suppose you write:

```
import "database/sql"
```

```
func main() {  
    db, err := sql.Open("sqlite3", "students.db")  
}
```

There is **no**

```
import _ "github.com/mattn/go-sqlite3"
```

Then the driver's `init()` never runs.

So the driver is never registered.

When `sql.Open()` looks for "sqlite3":

database/sql

Registered Drivers

(empty)

It says:

"I don't know any driver called sqlite3."

You'll get an error like:

sql: unknown driver "sqlite3" (forgotten import?)

---

## Why the underscore `_`?

Normally:

```
import "fmt"
```

means:

"I'm going to use `fmt`."

But

```
import _ "github.com/mattn/go-sqlite3"
```

means:

"I don't need to call anything from this package directly. I only want Go to load it and run its `init()`."

The `_` tells the compiler:

"Yes, I know I'm not using this package directly. Don't complain about an unused import."

---

## The complete startup flow

When your program starts:

main.go starts

↓

Imports database/sql

↓

Imports sqlite3 driver

↓

Go runs `sqlite3.init()`

↓

`sqlite3.init()` calls

`sql.Register("sqlite3", driver)`

↓

database/sql stores the driver

↓

main() starts

↓

sql.Open("sqlite3", ...)

Now `database/sql` knows exactly which driver to use.

---

## Think of it like registering for an event

Imagine `database/sql` is organizing a conference.

At first:

Speaker List

(empty)

The SQLite driver arrives and says:

"Hi, my name is `sqlite3`. Add me to your speaker list."

Now:

Speaker List

sqlite3

Later someone asks:

"Can I talk to `sqlite3`?"

The organizer replies:

"Yes, I know exactly who that is."

If the SQLite driver never registered itself, the organizer would say:

"Sorry, I've never heard of `sqlite3`."

---

This mechanism—**blank imports** (`_`) **plus automatic** `init()` **registration**—is used throughout Go, not just for databases. Once you understand it here, you'll recognize the same pattern in other libraries too.

Excellent question. This is actually about **design**, not databases. Let's build the intuition.

Suppose you have this:

```
type Sqlite struct {  
    Db *sql.DB  
}
```

Your question is:

**Why create a struct? Why not just use `*sql.DB` directly?**

Let's see.

---

## Option 1: No struct

Suppose you only have a database connection.

```
db, err := sql.Open("sqlite3", "students.db")
```

Now imagine you have a function:

```
func SaveStudent(db *sql.DB, name string) error {  
    // ...  
}
```

Every function needs the `db`.

```
SaveStudent(db, "Suyash")  
GetStudent(db, 1)
```

```
DeleteStudent(db, 5)
UpdateStudent(db, 3)
```

This works.

---

## But now suppose you're building a real application

You create a storage package.

```
storage/
  sqlite.go
```

Inside it:

```
type Sqlite struct {
    Db *sql.DB
}
```

Now instead of standalone functions, you write methods.

```
func (s *Sqlite) SaveStudent(name string) error {
    // use s.Db
}

func (s *Sqlite) GetStudent(id int) error {
    // use s.Db
}
```

Notice something?

You no longer pass the database every time.

The struct already has it.

---

# Think of it like a car

Imagine this:

```
type Car struct {  
    Engine Engine  
}
```

Would you write:

```
func Drive(engine Engine)
```

or

```
func (c *Car) Drive()
```

The second one.

Why?

Because the engine belongs to the car.

Similarly,

```
type Sqlite struct {  
    Db *sql.DB  
}
```

means

"A **Sqlite** object owns a database connection."

---

## Another reason: future expansion

Today you only have:

```
type Sqlite struct {  
    Db *sql.DB
```

```
}
```

Tomorrow you may need:

```
type Sqlite struct {  
    Db    *sql.DB  
    Logger *slog.Logger  
}
```

or

```
type Sqlite struct {  
    Db    *sql.DB  
    Logger *slog.Logger  
    Config Config  
}
```

or

```
type Sqlite struct {  
    Db    *sql.DB  
    Timeout time.Duration  
}
```

Now every method automatically has access to all of these.

---

## This is called encapsulation

The database connection is kept inside the object.

Instead of

SaveStudent(db, logger, config, timeout, ...)

you simply write

sqlite.SaveStudent(...)

because the `sqlite` object already contains everything it needs.

---

## This is also useful for interfaces

Suppose you have an interface:

```
type Storage interface {  
    SaveStudent(name string) error  
    GetStudent(id int) error  
}
```

Now your struct implements it:

```
type Sqlite struct {  
    Db *sql.DB  
}
```

with methods:

```
func (s *Sqlite) SaveStudent(name string) error { ... }
```

```
func (s *Sqlite) GetStudent(id int) error { ... }
```

This is a very common pattern in Go applications because it makes it easy to swap implementations (for example, SQLite, PostgreSQL, or an in-memory storage for tests) without changing the rest of your code.

---

## In your project

You're probably doing something like this:

```
type Sqlite struct {  
    Db *sql.DB  
}
```

```
func New(cfg Config) (*Sqlite, error) {  
    db, err := sql.Open("sqlite3", cfg.Path)  
    if err != nil {  
        return nil, err  
    }  
  
    return &Sqlite{  
        Db: db,  
    }, nil  
}
```

Then elsewhere:

```
storage, _ := sqlite.New(cfg)  
  
storage.SaveStudent(...)  
storage.GetStudent(...)  
storage.DeleteStudent(...)
```

The `storage` object **remembers** the database connection, so every method can use `s.Db` without you passing `db` around.

---

## The key idea

The struct is **not** required because of SQLite itself. It's created because **you want an object that groups together the database connection and all the methods that operate on it.**

This is a common Go design pattern: create a struct that holds shared dependencies (like `*sql.DB`), then define methods on that struct so those dependencies are available everywhere they're needed.

This is one of the **most important concepts** in backend engineering. If you understand this, you'll understand how almost every web application talks to databases.

Let's go step by step.

---

## First, what is a database connection?

Imagine your Go application wants to talk to a database.

It needs a communication channel.

Your Go App —————▶ PostgreSQL / SQLite / MySQL

That communication channel is called a **connection**.

Think of it like a phone call.

You 📞—————📞 Database

As long as the call is active, you can send SQL queries.

```
SELECT * FROM students;
```

The database sends back the results over that same connection.

---

## Can we create a new connection for every query?

Yes.

For example:

Open Connection

↓

Run Query

↓

Close Connection

Then again:

Open Connection

↓

Run Query



Close Connection

And again...

Open Connection



Run Query



Close Connection

Will this work?

✓ Yes.

Is it efficient?

✗ No.

---

## Why is opening a connection expensive?

Creating a connection isn't just one step.

Imagine connecting to PostgreSQL on another machine.

Go App



Find database IP



Create TCP connection



Authentication



Exchange protocol messages



Database accepts connection



Ready

This can take milliseconds.

That may not sound like much, but imagine your server handles:

- 1,000 requests per second
- or 10,000 requests per second

Creating thousands of new connections every second would waste a lot of time and resources.

---

## So what do we do instead?

We **reuse** existing connections.

Suppose your application starts.

Instead of opening and closing connections repeatedly, it keeps some open.

Database



Connection 1

▲  
|  
Connection 2

▲  
|  
Connection 3

Now, when a request arrives:

Request

↓

Need database

↓

Use Connection 2

↓

Run query

↓

Return Connection 2 to the pool

Notice:

The connection is **not closed**.

It goes back into the pool, ready for the next request.

---

## This is connection pooling

A **connection pool** is simply a collection of open database connections that your application reuses.

Think of it like a taxi stand.

Without pooling:

Need taxi

↓

Buy a new taxi

↓

Travel

↓

Destroy taxi

That would be ridiculous.

Instead:

Need taxi

↓

Take one from the taxi stand

↓

Travel

↓

Return it to the taxi stand

That's exactly what connection pooling does.

---

## Now let's read the documentation

It says:

DB is a database handle representing a pool of zero or more underlying connections.

Many beginners think:

```
db, _ := sql.Open(...)
```

creates one database connection.

**It does not.**

`db` is **not** a single connection.

It's a **manager**.

Imagine:

`db`

|

|— Connection 1

|— Connection 2

|— Connection 3

|— Connection 4

When you call:

```
db.Query(...)
```

`database/sql` picks one available connection from the pool, uses it, then returns it to the pool.

That's why the docs say `DB` is a **handle** to a pool, not a single connection.

---

## What does this mean?

It's safe for concurrent use by multiple goroutines.

Imagine your HTTP server.

Client A

Client B

Client C

Client D

Each request runs in its own goroutine.

Goroutine A

↓

`db.Query()`

At the same time:

Goroutine B

↓

`db.Exec()`

At the same time:

Goroutine C

↓

`db.Query()`

All of them use the same `*sql.DB`.

Is that okay?

☒ Yes.

Why?

Because `database/sql` internally coordinates access to the pool so goroutines don't accidentally use the same connection at the same time.

---

## What does this line mean?

The `sql` package creates and frees connections automatically.

You never write:

```
openConnection()
```

or

```
closeConnection()
```

`database/sql` decides:

- When to create a new connection.
- When to reuse an existing one.
- When to close idle connections.

It manages the pool for you.

---

## What are idle connections?

Suppose you have five open connections.

Pool

Connection 1 (busy)

Connection 2 (busy)

Connection 3 (idle)

Connection 4 (idle)

Connection 5 (idle)

Idle means:

The connection is open but currently not being used.

Keeping a few idle connections means future requests can start working immediately instead of paying the cost of creating a new connection.

---

## What is `SetMaxIdleConns()`?

Imagine you have:

100 idle connections

That's a lot of unused resources.

You can tell `database/sql`:

```
db.SetMaxIdleConns(10)
```

Now the pool keeps at most **10 idle connections**.

Extra idle connections are closed.

---

## What is a transaction (`Tx`)?

Normally:

```
db.Exec()
```

↓

Pool chooses any free connection

↓

Runs query

↓

Returns connection

Now suppose you start a transaction.

```
tx, err := db.Begin()
```

A transaction is a sequence of operations that must all happen together.

For example:

Withdraw ₹100

Deposit ₹100

If these ran on different connections or one succeeded while the other failed, your data could become inconsistent.

So when you call:

```
tx, _ := db.Begin()
```

`database/sql` takes **one specific connection** from the pool and reserves it for that transaction.

Pool

Connection 1

Connection 2 ← Reserved for tx

Connection 3

Every query through `tx` uses that same connection until you finish.

When you call:

tx.Commit()

or

tx.Rollback()

the connection is released back into the pool.

That's why the documentation says:

Once `DB.Begin` is called, the returned `Tx` is bound to a single connection.

---

## The biggest misconception to avoid

Many beginners think:

```
db, _ := sql.Open(...)
```

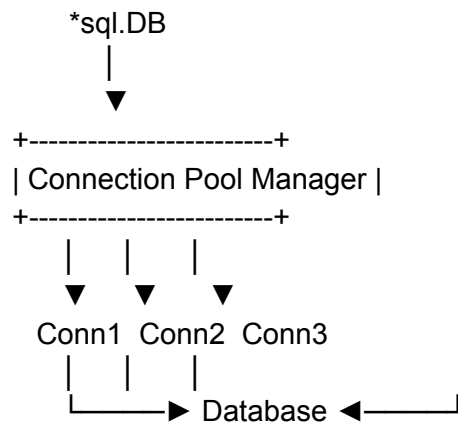
means:

db

is one database connection.

**That's incorrect.**

Think of it like this instead:



`*sql.DB` is the **manager of the pool**, not the connection itself. Every time you execute a query, it borrows a connection from the pool, uses it, and then returns it when the operation is complete (unless it's reserved for something like a transaction). This design lets many goroutines safely share the same `*sql.DB` while efficiently reusing database connections.

Excellent question. **Prepared statements** are another concept that confuses many developers at first. Let's build it from the ground up.

---

## Imagine you keep asking the database the same query

Suppose your code does this:

```
db.Query("SELECT * FROM students WHERE id = ?", id)
```

Today:

```
SELECT * FROM students WHERE id = 1;
```

Tomorrow:

```
SELECT * FROM students WHERE id = 5;
```

Later:

```
SELECT * FROM students WHERE id = 10;
```

Notice something?

The **SQL query itself never changes**.

Only the value changes.

---

## What does the database do every time?

Every time you send SQL, the database has to:

Receive SQL

↓

Parse it

↓

Check if the SQL is valid

↓

Create an execution plan

↓

Execute it

Even though it's the same query over and over.

---

## What is a Prepared Statement?

A prepared statement says to the database:

"I'm going to use this SQL many times. Please prepare it once."

Like this:

```
stmt, err := db.Prepare(  
    "SELECT * FROM students WHERE id = ?",  
)
```

Now the database:

Receives SQL

↓

Parses it



Creates execution plan



Stores it

Now whenever you need it:

`stmt.Query(1)`

or

`stmt.Query(5)`

or

`stmt.Query(10)`

the database can reuse the prepared version instead of starting from scratch each time.

---

## Real-world analogy

Imagine you're ordering coffee every morning.

Without preparing:

Customer:

Hi...

I want...

A medium cappuccino...

with oat milk...

extra hot...

please.

Every day.

---

Instead, the café says:

We'll save your usual order.

Tomorrow you just say:

Usual order.

Much faster.

That's exactly what a prepared statement is.

---

## Why do we pass parameters separately?

Instead of:

```
db.Query(  
    "SELECT * FROM students WHERE id = 5",  
)
```

you prepare:

```
stmt, _ := db.Prepare(  
    "SELECT * FROM students WHERE id = ?",  
)
```

Then:

```
stmt.Query(5)
```

The SQL stays the same.

Only the value changes.

---

## Another huge advantage: SQL Injection protection

Suppose someone enters:

1 OR 1=1

If you build SQL yourself:

```
query := "SELECT * FROM students WHERE id = " + userInput
```

You might accidentally execute:

```
SELECT * FROM students  
WHERE id = 1 OR 1=1;
```

Oops.

Every row comes back.

---

Prepared statements separate:

- SQL code
- Data

The database never treats the parameter as SQL code.

It treats it only as a value.

This is why prepared statements are an important defense against SQL injection.

---

## What is `*sql.Stmt`?

When you call:

```
stmt, err := db.Prepare(...)
```

`stmt` is an object representing the prepared query.

You can reuse it.

```
stmt.Query(1)
```

```
stmt.Query(2)
```

```
stmt.Query(3)
```

```
stmt.Query(100)
```

---

## Why do we call `stmt.Close()`?

Look at what happened earlier.

The database stored:

Prepared Statement

↓

Execution Plan

↓

Memory

The database is keeping resources for that statement.

If you're finished with it, you should tell the database:

"I'm done. You can free those resources."

That's what:

```
stmt.Close()
```

does.

---

## Why `defer stmt.Close()`?

Suppose you write:

```
stmt, err := db.Prepare(...)
if err != nil {
    return err
}
```

Now imagine later:

```
rows, err := stmt.Query(id)
if err != nil {
    return err
}
```

If you forget to close the statement:

Prepared statement

↓

Still allocated

↓

Memory not released

If your program creates lots of prepared statements and never closes them, those resources can accumulate unnecessarily.

---

Using `defer` solves this neatly:

```
stmt, err := db.Prepare(...)
if err != nil {
    return err
}
```

```
defer stmt.Close()
```

Now no matter how the function exits:

- normal return ✓
- early return because of an error ✓
- any other return path ✓

Go will automatically execute:

```
stmt.Close()
```

just before the function returns.

---

## Timeline

```
stmt, _ := db.Prepare(...)
```

Prepared Statement Created

↓

```
defer stmt.Close()
```

Close scheduled

↓

stmt.Query(...)

↓

stmt.Exec(...)

↓

Function finishes

↓

Go automatically runs

stmt.Close()

↓

Database frees resources

---

## One more important thing

For many applications, you **don't need to call `Prepare()` manually**.

When you do:

```
db.Query("SELECT * FROM students WHERE id = ?", id)
```

or

```
db.Exec("INSERT INTO students(name) VALUES(?)", name)
```

many database drivers already use prepared statements internally or optimize repeated execution for you.

So manual `Prepare()` is most useful when **you know you'll execute the exact same SQL many times** and want to reuse the same prepared statement yourself.

---

## Summary

- `Prepare()` sends the SQL to the database once so it can parse and plan it in advance.
- It returns a `*sql.Stmt`, which represents that reusable prepared statement.
- You call `stmt.Query(...)` or `stmt.Exec(...)` multiple times with different parameter values.
- `stmt.Close()` tells the database you're finished with the prepared statement so it can release associated resources.
- `defer stmt.Close()` ensures those resources are released even if the function returns early because of an error.

**Short answer: The resources are primarily on the *database server*.**

When you do:

```
stmt, err := db.Prepare("SELECT * FROM students WHERE id = ?")
```

The flow is:

Your Go code



database/sql



Driver



Database

The **driver** sends the `PREPARE` request to the **database**.

The **database** stores the prepared statement (execution plan, metadata, etc.).

So when you call:

```
defer stmt.Close()
```

you're telling the **database**:

"I'm done with this prepared statement. You can free the resources."

The driver simply forwards that request to the database.

This is a fantastic question. You're now asking **how a database actually thinks internally**, not just how to use it. Let's simplify what's happening inside.

---

## Suppose you send this SQL

```
SELECT * FROM students WHERE id = 10;
```

The database **doesn't execute the text directly**.

It first understands what the text means.

Think of it like a compiler for programming languages.

---

### Step 1: Receive SQL

The database receives a string over the network.

```
"SELECT * FROM students WHERE id = 10;"
```

At this point, it's just text.

---

### Step 2: Parse

The parser breaks the SQL into pieces (called **tokens**).

```
SELECT  
*  
FROM  
students  
WHERE  
id  
=  
10
```

Then it builds an internal tree representing the query.

Very roughly:

```
SELECT
  |
FROM students
  |
WHERE
  |
id = 10
```

The database is no longer working with the original string. It's working with an internal representation.

---

## Step 3: Validate

The database asks questions like:

Does the table "students" exist?

Does the column "id" exist?

Can the current user read this table?

Is the SQL syntax correct?

If something is wrong:

```
SELECT xyz FROM students;
```

and **xyz** doesn't exist, it returns an error.

---

## Step 4: Create an execution plan

This is the interesting part.

Suppose your table has **10 million rows**.

You ask:

```
SELECT * FROM students WHERE id = 10;
```

How should the database find row 10?

There are multiple ways.

### **Option 1: Full table scan**

Row 1

Row 2

Row 3

...

Row 10,000,000

Very slow.

---

### **Option 2: Use an index**

Imagine the table has an index on **id**.

Instead of reading every row:

Index

1 -> row location

2 -> row location

3 -> row location

10 -> row location

The database can jump directly to the correct row.

Much faster.

---

The database chooses the cheapest strategy.

That chosen strategy is called the **execution plan**.

A simplified plan might look like:

Use index on id

↓

Find row

↓

Return result

Or, if there's no index:

Read every row

↓

Compare id

↓

Return matching row

---

## Step 5: Execute

Now the database follows the plan.

If the plan says:

Use index

it uses the index.

If the plan says:

Scan table

it scans the table.

---

## Now what does a prepared statement store?

Suppose you prepare:

```
SELECT * FROM students WHERE id = ?
```

Notice something important.

The SQL has a **placeholder**.

Not:

```
id = 10
```

Instead:

```
id = ?
```

---

The database stores something like:

SQL Template

```
SELECT * FROM students  
WHERE id = ?
```

and also stores the compiled internal representation and execution plan.

Conceptually:

Prepared Statement

SQL Template

↓

Parsed Tree

↓

Execution Plan

↓

Stored in memory

---

## Does it store the value?

**No.**

This is the key idea.

When you prepare:

```
SELECT * FROM students WHERE id = ?
```

there is **no value yet**.

The database stores only the **template**.

Later you execute:

```
stmt.Query(10)
```

The database takes the prepared template:

```
SELECT * FROM students  
WHERE id = ?
```

and binds:

? = 10

Next time:

stmt.Query(25)

It uses the **same prepared statement**.

Only the parameter changes.

? = 25

The SQL template stays exactly the same.

---

## Think of it like a cake mold

The prepared statement is the mold.

Cake Mold

[   ]

Today:

Pour chocolate batter.

Tomorrow:

Pour vanilla batter.

The mold doesn't change.

Only what's poured into it changes.

Similarly:

Prepared SQL

```
SELECT ...  
WHERE id = ?
```

Today:

? = 10

Tomorrow:

? = 42

The prepared statement remains the same; only the parameter value changes.

---

## The key takeaway

A prepared statement **does not store**:

```
SELECT * FROM students WHERE id = 10;
```

or

```
SELECT * FROM students WHERE id = 25;
```

Instead, it stores a reusable form of:

```
SELECT * FROM students WHERE id = ?
```

along with the database's internal parsed representation and (where applicable) an execution plan. Each time you execute it, you supply new parameter values, and the database plugs those values into the already-prepared query without reparsing the SQL text. This reuse is what makes prepared statements both efficient and safe against SQL injection.

**YES! Exactly.** You're understanding it correctly. 👍

The **execution plan** is basically the database's strategy for answering your query.

For example, for:

```
SELECT * FROM students WHERE id = ?
```

the execution plan might be:

Use PRIMARY KEY index



Find row with id = ?



Return row

Or, if there's **no index**:

Scan entire students table



Compare every row's id



Return matching row

Or for a JOIN query:

Read students table



Join with courses table



Sort results



Return rows

So yes, an execution plan is a sequence of operations like:

- ☒ Use an index
- ☒ Scan the whole table
- ☒ Join two tables
- ☒ Sort rows
- ☒ Filter rows
- ☒ Aggregate (**COUNT**, **SUM**, etc.)

---

## What is stored in memory?

Suppose you do:

```
stmt, _ := db.Prepare(  
    "SELECT * FROM students WHERE id = ?"  
)
```

The database stores something conceptually like this:

Prepared Statement

-----

SQL Template:

```
SELECT * FROM students  
WHERE id = ?
```

Parsed Tree:

```
SELECT  
  FROM students  
  WHERE id = ?
```

Execution Plan:

Use PRIMARY KEY index

↓

Fetch row

↓

Return row

Notice what's **NOT** stored:

✗

```
SELECT * FROM students WHERE id = 10;
```

or

```
SELECT * FROM students WHERE id = 20;
```

The values (10, 20, etc.) are **not** part of the prepared statement.

When you later execute:

```
stmt.Query(10)
```

the database takes the stored plan and temporarily binds:

? = 10

When you execute:

```
stmt.Query(20)
```

it reuses the **same** plan and simply binds:

? = 20

So, in short:

- **Stored in memory:**
  - ☒ SQL template (`SELECT ... WHERE id = ?`)
  - ☒ Parsed internal representation
  - ☒ Execution plan
- **Not stored:**
  - ☒ Parameter values like `10, 20, 30`

---

One small technical note: modern databases (like PostgreSQL) don't always lock in a single execution plan forever. Depending on the query and parameter values, they may reuse the prepared plan or generate a different one if it's likely to perform better. But the overall idea you've learned is the right mental model.

Excellent question. **No, a prepared statement is *not* a stored procedure.** They are related, but they are different concepts.

Let's compare them.

### Prepared Statement

Stores a SQL query

### Stored Procedure

Stores a program (multiple SQL statements + logic)

Usually created by the application

Created inside the database

Lives for the duration of the connection/session (typically)

Lives in the database until you drop it

Reuses parsing/execution plan

Can contain variables, loops, conditions, transactions, etc.

---

## Prepared Statement

Suppose you do:

```
stmt, _ := db.Prepare(  
    "SELECT * FROM students WHERE id = ?"  
)
```

The database stores something like:

SQL Template

```
SELECT * FROM students  
WHERE id = ?
```

Later:

```
stmt.Query(10)  
stmt.Query(20)  
stmt.Query(30)
```

It reuses that prepared statement.

---

## Stored Procedure

A stored procedure is like writing a function **inside the database**.

For example (conceptually):

```
CREATE PROCEDURE GetStudent(IN studentId INT)
```

```
BEGIN
  SELECT * FROM students
  WHERE id = studentId;
END;
```

Later you call:

```
CALL GetStudent(10);
```

The procedure already exists in the database.

---

## Think of it like programming

### Prepared Statement

Like this in Go:

```
func Query(id int) {
    // Same SQL, different values
}
```

It's just one reusable query.

---

### Stored Procedure

Like this:

```
func TransferMoney(from, to int, amount float64) {
    // Validate account
    // Check balance
    // Withdraw
    // Deposit
    // Log transaction
}
```

It contains **logic**, not just one SQL statement.

---

## Easy way to remember

### Prepared Statement

"Please remember this SQL query."

### Stored Procedure

"Please remember this entire program."

So when you call:

```
stmt, _ := db.Prepare(...)
```

you are **not creating a stored procedure**. You're asking the database to prepare one SQL statement so it can be executed efficiently multiple times.

Of course!

```
type Sqlite struct {  
    Db *sql.DB  
}
```

### What is **Db \*sql.DB**?

- **Db** → the **field name** in your struct.
- **\*sql.DB** → the **type** of that field.

So you're saying:

"This struct has a field named **Db** that stores a pointer to an **sql.DB**."

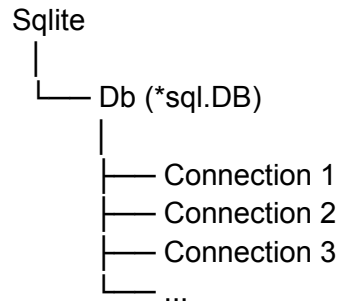
---

### But what is **\*sql.DB**?

Despite its name, **\*sql.DB** is **NOT** a single database connection.

It is a **database handle** (or manager) that manages a **pool of database connections**.

Think of it like this:



---

## Why keep it in the struct?

So every method can use the same database handle.

```
func (s *Sqlite) SaveStudent() {  
    s.Db.Exec(...)  
}
```

```
func (s *Sqlite) GetStudent() {  
    s.Db.Query(...)  
}
```

Both methods use the same `s.Db`.

---

## One-line summary

**`Db *sql.DB` is a field that stores a pointer to Go's database handle, which manages a pool of database connections and is shared by all methods of the struct.**

**Yes**, with one small correction.

The **connection is between your application (through the driver) and the database.**

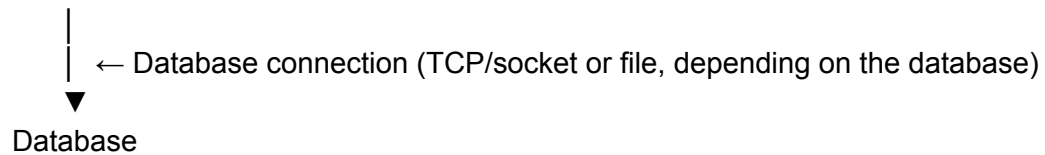
Your Go Code



database/sql



Driver



- `database/sql` manages the pool of connections.
- The **driver** uses those connections to communicate with the database.
- The **database** receives requests over those connections.

So if you're asking:

"Is the database connection between the driver and the database?"

Yes. ✓

Just remember that `database/sql` owns and manages those connections, while the **driver** actually uses them to send and receive data.

This is the right question. You're asking **how connection pooling works internally**, not just what it is.

Let's imagine you're using **PostgreSQL** (the idea is the same for MySQL).

---

## Without connection pooling

Suppose a request comes in.

Client

↓

Go Application

↓

Open TCP connection

↓

Authenticate

↓

Run SQL

↓

Close TCP connection

The connection dies.

Now the next request comes.

Again:

Open TCP connection

↓

Authenticate

↓

Run SQL

↓

Close TCP connection

You're doing expensive work every time.

---

## With **\*sql.DB**

When your application starts:

```
db, err := sql.Open("postgres", connString)
```

Think of **db** as saying:

"I'll manage connections."

Initially, there might be **0** actual connections.

\*sql.DB

Pool

(empty)

---

## First query

You write:

db.Query(...)

database/sql looks in its pool.

Pool

(empty)

No connection?

It tells the driver:

"Please create one."

The driver does this:

Driver

↓

Open TCP connection

↓

Authenticate

↓

Connected

Now:

Pool

Connection #1

The query runs.

---

## Here's the important part

When the query finishes...

**It does NOT close the connection.**

Instead:

Connection #1

↓

Returned to pool

↓

Waiting...

The TCP connection is still alive.

It's just sitting there, idle.

---

## Second query arrives

Again:

db.Query(...)

database/sql looks:

Pool

Connection #1 (idle)

Instead of creating a new connection:

Take Connection #1

↓

Run query

↓

Return Connection #1

Same connection.

No reconnect.

No authentication again.

Much faster.

---

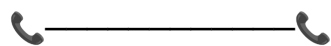
## How does it "keep it alive"?

This is the key.

A TCP connection stays alive **until someone closes it**.

Imagine a phone call.

You call your friend.



You talk.

You stop talking for 10 seconds.

Is the call disconnected?

**No.**

The phone call is still active.

Similarly:

Go App ===== Database

The TCP socket stays open.

When you're not using it, it's just **idle**.

---

## Who closes the connection?

Eventually:

- the application exits
- the database closes it
- it times out
- `database/sql` decides it has too many idle connections
- you call:

`db.Close()`

Only then is the connection actually closed.

---

## Internally, imagine something like this

This is **not** the real Go code, but it gives you the idea.

```
type DB struct {  
    idleConnections []Connection  
}
```

When you do:

```
db.Query(...)
```

Internally:

```
if there is an idle connection {  
    use it  
} else {  
    create a new one  
}
```

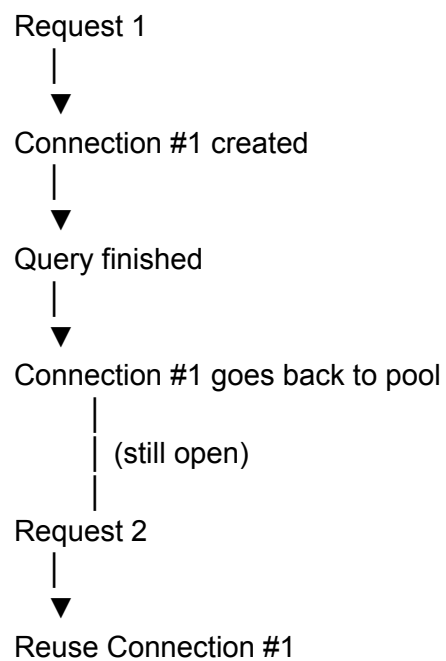
After the query:

```
return connection back to idleConnections
```

It doesn't destroy it.

---

## Real picture



↓  
Return to pool

Notice that **the connection never dies between Request 1 and Request 2.**

---

## The most important thing to remember

A database connection is just a live TCP connection (or similar transport, depending on the database) between your application and the database server.

`database/sql` doesn't do anything magical to "freeze" it. It simply **chooses not to close it** after each query. Instead, it keeps the open connection in an idle pool so the next query can reuse it.

That's the essence of **connection pooling**.

Excellent question. This is exactly the kind of thinking backend engineers need.

Suppose your server receives **10,000 requests** at once.

Does `database/sql` create **10,000 database connections**?

**No.** It tries to reuse connections and obeys the limits you configure.

Let's say you've configured:

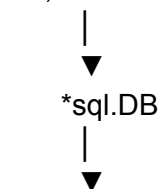
```
db.SetMaxOpenConns(100)
```

This means:

"At most 100 database connections may be open at the same time."

Now imagine:

10,000 HTTP Requests



Connection Pool (max 100)

## What happens?

- The **first 100 requests** each get one of the available connections.
- The **remaining 9,900 requests** wait until a connection is returned to the pool.

As soon as one request finishes:

Request 1 finishes



Connection returned to pool



Request 101 gets that connection

Then:

Request 2 finishes



Connection returned



Request 102 uses it

The connections keep getting **reused**.

---

## Why not open 10,000 connections?

Because that's usually a bad idea.

Every connection consumes resources:

- Memory in your application
- Memory in the database server
- CPU for managing the connection
- Network/socket resources

If 10,000 clients all opened connections at once, the database server could become overloaded.

That's why connection pools exist: they **limit concurrency at the database layer** while allowing your application to handle many more requests overall.

---

## A real-world analogy

Imagine a restaurant:

- **10,000 customers** arrive.
- The restaurant has **100 tables**.

Only **100 customers** can eat at a time.

When one table becomes free, the next customer is seated.

The tables are like **database connections**.

The customers are like **HTTP requests**.

---

## One important clarification

Not every request necessarily needs the database.

For example:

- `GET /health` → may not use the database.
- Returning cached data → may not use the database.
- Serving a static file → may not use the database.

So even if you receive 10,000 HTTP requests, the number of **simultaneous database queries** is often much smaller.

This is one reason a pool of a few dozen or a few hundred connections is enough for many high-traffic applications.

You're asking exactly the right question. The confusion comes from mixing up the responsibilities of **database/sql** and the **driver**. Let's separate them.

## Step 1: You call

```
db, err := sql.Open("sqlite3", "students.db")
```

You might think:

"This opens a database connection."

**Not exactly.**

`sql.Open()` mainly creates a `*sql.DB` object, which is the **connection pool manager**.

Think of it as:

`sql.Open()`

↓

Creates `*sql.DB`

↓

`*sql.DB` says:

"I know which driver to use.

I'll manage connections."

At this point, **there may be zero actual database connections**.

---

## Step 2: First query

Now you do:

`db.Query("SELECT * FROM students")`

`*sql.DB` checks its pool.

Pool

(empty)

It says:

"I need a connection."

So `database/sql` asks the driver:

"Please open one connection."

The driver does the actual work:

Driver

↓

Open TCP connection (or SQLite file)

↓

Authenticate (if needed)

↓

Return connection

---

## Who manages the pool?

`database/sql` ✓

It decides:

- How many connections to keep.
  - When to reuse one.
  - When to create a new one.
  - When to close idle ones.
- 

## Who actually opens a connection?

The driver. ✓

The driver knows:

- PostgreSQL protocol
- MySQL protocol
- SQLite file format

`database/sql` doesn't know how to talk to PostgreSQL or SQLite directly.

---

## Then why does `sql.Open()` exist?

Because `database/sql` needs to know **which driver** to use.

```
sql.Open("sqlite3", ...)
```

means:

"Use the registered `sqlite3` driver."

Internally, it's roughly like:

`database/sql`

↓

Find driver named "sqlite3"

↓

Store it inside `*sql.DB`

---

## Does `sql.Open()` tell the driver to create many connections?

**No.**

This is the important point.

When you call:

```
db, _ := sql.Open(...)
```

it does **not** say:

"Create 100 connections."

Instead, it creates the manager.

Later, when queries arrive:

Query 1



Need connection



Ask driver to create one



Pool has 1 connection

Then:

Query 2



Reuse existing connection

Or, if the first connection is busy:

Need another connection



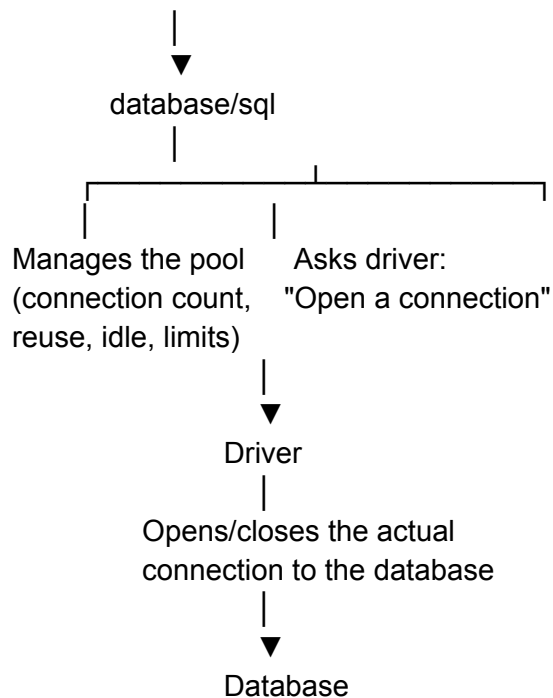
Ask driver to create Connection #2

Connections are usually created **on demand**, not all at once.

---

## The responsibility split

Your Go Code



## The one sentence to remember

- **database/sql** is the manager.
- The driver is the worker.

The manager decides **when** a connection is needed.

The worker knows **how** to create and use that connection. That's the mental model professional Go developers use.

**Yes! Exactly.** ✓

Think of it like this:

database/sql = Manager 🧑

Driver = Worker 🧑

Database = Server 🖥️

The **manager** (**database/sql**) decides:

- ✓ Create a new connection.
- ✓ Reuse an idle connection.
- ✓ Return a connection to the pool.
- ✓ Close an idle connection.

-  Limit the maximum number of open connections.
-  Limit the maximum number of idle connections.

But the manager **doesn't physically open or close the network connection**.

Instead, it tells the driver:

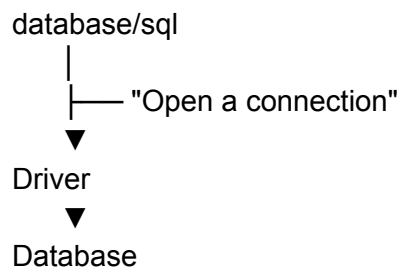
"Please open a connection."

or

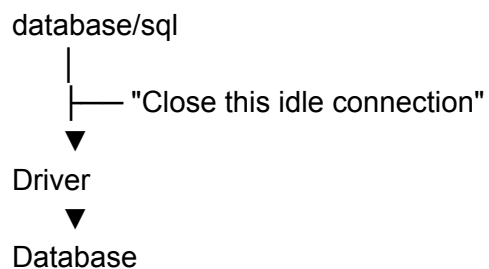
"Please close this connection."

The **driver** then performs the actual operation.

So the flow is:



Later:



## One sentence to remember

**database/sql** decides *when* to open or close a connection, while the driver knows *how* to open or close that connection.

That's the clean separation of responsibilities.